

Conversor AFN para AFD

Estruturas de Dados e Algoritmo Principal

Documentacao Tecnica - Henrique

Henrique Freitas

Dezembro 2025

Sumário

1	Introducao	3
1.1	Visao Geral do Projeto	3
2	Parte 1: Estruturas de Dados e Interface (L1-2254)	4
2.1	Visao Geral da Parte 1	4
2.2	Tipos Fundamentais (L39-50)	4
2.2.1	TTransition - Record de Transicao (L39-43)	4
2.2.2	TTransitionArray - Array Dinamico (L49)	4
2.3	Classe TFormMain - Formulario Principal (L50-134)	5
2.3.1	Declaracao da Classe (L76-130)	5
2.4	Variaveis Globais dos Automatos (L135-165)	5
2.5	Declaracoes de Metodos Privados (L166-298)	6
2.5.1	RemoveEpsilonTransitions (L174)	6
2.5.2	ConvertAFNtoAFD (L190)	6
2.5.3	MinimizeDFA (L207)	6
2.5.4	Metodos Auxiliares (L222-298)	7
2.6	TStringSet - Classe de Conjuntos (L299-475)	7
2.6.1	Declaracao da Classe (L299-311)	7
2.6.2	Implementacao dos Metodos (L333-475)	8
2.7	Inicializacao da Interface (L488-571)	8
2.8	Parser JSON para TXT (L693-918)	8
2.9	LoadSampleFile - Exemplo Padrao (L952-1043)	8
2.10	btnLoadFileClick - Carregar Arquivo (L1085-1170)	9
2.11	btnClearClick - Limpar Interface (L1237-1360)	9
2.12	btnRemoveEpsilonClick - Handler Epsilon (L1395-1566)	9
2.13	btnUseAsInputClick - Copiar Resultado (L1603-1720)	9
2.14	btnConvertClick - Handler Conversao (L1863-2068)	10
3	Parte 2: Conversao AFN para AFD (L2958-3795)	11
3.1	Teoria: Construcão de Subconjuntos	11
3.1.1	A Grande Ideia	11
3.1.2	Motivacao: Por Que Precisamos Disto?	11
3.1.3	Por Que TStringSet e Essencial?	11

3.2	Estrutura do Algoritmo ConvertAFNtoAFD (L2958-3795)	11
3.3	Declaracao de Variaveis (L2959-2978)	12
3.4	Limpeza e Inicializacao (L2980-3005)	12
3.5	Validacao de Entrada (L3007-3018)	13
3.6	Parse do Alfabeto (L3033-3058)	13
3.7	Parse dos Estados (L3067-3075)	13
3.8	Parse dos Estados Iniciais (L3084-3092)	13
3.9	Parse dos Estados Finais (L3101-3109)	14
3.10	Parse das Transicoes (L3130-3177)	14
3.11	Aviso de Epsilon (L3179-3191)	14
3.11.1	Filosofia de Validacao: Avisar vs Bloquear	14
3.12	Logging da Entrada (L3206-3217)	15
3.13	Criacao das Estruturas BFS (L3251-3257)	15
3.13.1	Por Que BFS e Nao DFS?	15
3.13.2	Por Que OwnsObjects = True?	16
3.14	Estado Inicial do AFD (L3273-3283)	16
3.15	Loop Principal BFS (L3296-3540)	16
3.15.1	Exemplo Pratico Passo-a-Passo	17
3.16	Remocao da Fila (L3308-3311)	18
3.17	Verificacao de Estado Final (L3326-3330)	18
3.18	Loop de Simbolos (L3351-3525)	18
3.19	Calculo de Destinos (L3356-3373)	19
3.20	Tratamento de Conjunto Vazio (L3375-3379)	19
3.20.1	Por Que Nao Adicionar Estado de Erro?	19
3.21	Registro de Transicao (L3395-3409)	20
3.22	Descoberta de Novos Estados (L3426-3439)	20
3.22.1	Gerenciamento de Memoria Detalhado	20
3.22.2	Por Que IndexOf ao Inves de Contains?	21
3.23	Formatacao da Saida (L3549-3638)	21
3.23.1	Por Que Dois Destinos para o Resultado?	21
3.23.2	Formato de Estados do AFD	22
3.24	Habilitacao da Interface (L3640-3647)	23
3.25	Limpeza de Memoria (L3650-3665)	23
3.26	Paint Handlers - Desenho de Diagramas (L3698-3792)	23
3.26.1	PaintBoxEpsilonNFAPaint (L3698-3710)	23
3.26.2	PaintBoxNFAPaint (L3712-3718)	24
3.26.3	PaintBoxDFAPaint (L3720-3741)	24
3.26.4	PaintBoxMinDFAPaint (L3743-3764)	24
3.27	Complexidade do Algoritmo Completo	24
3.28	Mapa Completo de Funcoes	25
4	Conclusao	26
4.1	Resumo das Responsabilidades	26
4.2	Pontos-Chave para Apresentacao	26
4.3	Complexidade do Algoritmo	26

1 Introducao

Este documento detalha as secoes do codigo `MainForm.pas` sob responsabilidade de **Henrique**:

- **Parte 1:** Linhas 1-2254 (Estruturas de Dados, Interface, Parsing)
- **Parte 2:** Linhas 2959-3795 (Algoritmo AFN para AFD)

1.1 Visao Geral do Projeto

O projeto implementa um conversor completo de automatos finitos:

1. **AFN- ϵ** (com epsilon) para **AFN** (sem epsilon)
2. **AFN** (nao-deterministico) para **AFD** (deterministico)
3. **AFD** para **AFD Minimizado**

Tecnologia: Free Pascal + Lazarus (GUI nativa)

2 Parte 1: Estruturas de Dados e Interface (L1-2254)

2.1 Visao Geral da Parte 1

A Parte 1 contem:

- Tipos fundamentais (TTransition, TTransitionArray)
- Classe TStringSet (L299-475)
- Variaveis globais (L135-150)
- FormCreate - Inicializacao (L488-571)
- LoadTestFiles - Scan de arquivos (L586-643)
- OnTestFileSelected - Handler ComboBox (L645-679)
- ConvertJSONToText - Parse arquivo JSON (L693-812)
- ConvertJSONTextToText - Parse string JSON (L829-918)
- LoadSampleFile - Exemplo padrao (L952-1043)
- btnLoadFileClick - Carregar arquivo (L1085-1170)
- btnClearClick - Limpar interface (L1237-1360)
- btnRemoveEpsilonClick - Handler epsilon (L1395-1566)
- btnUseAsInputClick - Copiar resultado (L1603-1720)
- btnConvertClick - Handler conversao (L1863-2068)

2.2 Tipos Fundamentais (L39-50)

2.2.1 TTransition - Record de Transicao (L39-43)

Referencia: MainForm.pas linhas 39-43

Listing 1: Definicao de TTransition

```
1 TTransition = record
    FromState: string;    (* Estado de origem *)
    Symbol: string;       (* Simbolo da transicao *)
    ToState: string;      (* Estado de destino *)
end;
```

Explicacao:

- Representa uma transicao: $\delta(q_i, a) = q_j$
- FromState = q_i , Symbol = a , ToState = q_j

2.2.2 TTransitionArray - Array Dinamico (L49)

```
TTransitionArray = array of TTransition;
```

Array dinamico que armazena todas as transicoes do automato.

2.3 Classe TFormMain - Formulário Principal (L50-134)

Referencia: MainForm.pas linhas 50-134

2.3.1 Declaracao da Classe (L76-130)

A classe TFormMain herda de TForm e gerencia toda a interface grafica:

Componentes Visuais:

- **Botoes:** btnConvert, btnLoadFile, btnClear, btnMinimize, btnRemoveEpsilon, btnUseAsInput
- **Editores de texto:** memoInput, memoOutput, memoNFAOutput, memoMinOutput
- **Paineis de desenho:** PaintBoxEpsilonNFA, PaintBoxNFA, PaintBoxDFA, PaintBoxMinDFA
- **Abas:** TabEpsilonNFA, TabNFA, TabDFA, TabMinDFA
- **Dialogo:** OpenFileDialog (para selecionar arquivos)

Event Handlers:

- btnClearClick - Limpar interface
- btnConvertClick - Converter AFN para AFD
- btnLoadFileClick - Carregar arquivo
- btnMinimizeClick - Minimizar AFD
- btnRemoveEpsilonClick - Remover epsilon-transicoes
- btnUseAsInputClick - Copiar resultado para entrada
- FormCreate - Inicializacao do formulario
- PaintBoxXXXPaint - Desenhar diagramas

2.4 Variaveis Globais dos Automatos (L135-165)

Referencia: MainForm.pas linhas 135-165

Listing 2: Variaveis Globais

```
(* AFN-epsilon: entrada original *)
EpsilonNFAAlphabet, EpsilonNFASStates: TStringList;
EpsilonNFAInitials, EpsilonNFAFinals: TStringList;
4 EpsilonNFATransitions: TTransitionArray;

(* AFN: sem epsilon *)
NFAAlphabet, NFASStates, NFAInitials, NFAFinals: TStringList;
NFATransitions: TTransitionArray;
9

(* AFD: deterministico *)
DFAStates, DFAFinals: TStringList;
```

```

DFAInitial: string;  (* AFD tem apenas UM inicial *)
DFATransitions: TTransitionArray;

14  (* AFD Minimizado *)
MinDFAStates, MinDFAFinals: TStringList;
MinDFAInitial: string;
MinDFATransitions: TTransitionArray;

```

2.5 Declaracoes de Metodos Privados (L166-298)

Referencia: MainForm.pas linhas 166-298

A secao privada da classe declara os metodos principais de conversao:

2.5.1 RemoveEpsilonTransitions (L174)

Proposito: Remove epsilon-transicoes de um AFN- ε

Algoritmo:

1. Calcula epsilon-fecho (ε -closure) de cada estado
2. Para cada estado e simbolo, adiciona transicoes para todos destinos alcancaveis
3. Ajusta estados finais (estado e final se seu ε -fecho contem algum final)

Complexidade: $O(n^2 \cdot m)$ onde n = estados, m = transicoes

2.5.2 ConvertAFNtoAFD (L190)

Proposito: Converte AFN em AFD usando construcao de subconjuntos

Algoritmo (Subset Construction):

1. Estado inicial do AFD = conjunto dos estados iniciais do AFN
2. Para cada estado do AFD (conjunto de estados do AFN):
 - Para cada simbolo do alfabeto:
 - Calcula conjunto de estados alcancaveis
 - Cria novo estado do AFD se necessario
 - Adiciona transicao
3. Estado do AFD e final se contem algum estado final do AFN

Complexidade: $O(2^n \cdot |\Sigma|)$ no pior caso (explosao exponencial)

2.5.3 MinimizeDFA (L207)

Proposito: Minimiza um AFD usando algoritmo de Myhill-Nerode

Algoritmo (Particao-Refinamento):

1. Particao inicial: (finais, nao-finais)
2. Repetir ate nao haver mudancas:

- Para cada particao P e simbolo a:
- Se estados de P vao para particoes diferentes com 'a', divide P

3. Cada particao final vira um estado do MinDFA

Complexidade: $O(n \cdot m \cdot \log n)$ onde n = estados, m = transicoes

2.5.4 Metodos Auxiliares (L222-298)

LoadSampleFile: Carrega arquivo de exemplo `sample_afn.txt`

LoadTestFiles: Popula ComboBox com arquivos do diretorio `testes/`

OnTestFileSelected: Handler para selecao de arquivo de teste

ConvertJSONToText: Converte arquivo JSON para formato TXT

ConvertJSONTextToText: Converte string JSON para formato TXT

DrawAutomaton: Desenha diagrama de automato em TCanvas

- Layout: 4 estados por linha
- Espacamento: 100px horizontal, 80px vertical
- Estados: circulos de raio 20px
- Estados finais: circulo duplo com fundo amarelo
- Transicoes: linhas com setas e rotulos

2.6 TStringSet - Classe de Conjuntos (L299-475)

Referencia: MainForm.pas linhas 299-475

2.6.1 Declaracao da Classe (L299-311)

Listing 3: Classe TStringSet

```

TStringSet = class
2   private
      FList: TStringList;
   public
      constructor Create;
      destructor Destroy; override;
7   procedure Add(const s: string);
      function Contains(const s: string): Boolean;
      function ToString: string; override;
      function Clone: TStringSet;
      function IsEmpty: Boolean;
12  function Count: Integer;
      function Item(i: Integer): string;
end;
```

Por Que TStringSet?

Construcao de subconjuntos gera muitos conjuntos de estados. Precisamos evitar duplicatas. TStringList com `Duplicates := dupIgnore` resolve!

2.6.2 Implementacao dos Metodos (L333-475)

Create (L333-337): Inicializa com Sorted=True e Duplicates=dupIgnore

Destroy (L339-343): Libera FList

Add (L360-367): Adiciona elemento (ignora duplicatas automaticamente)

Contains (L377-381): Verifica se elemento existe ($O(\log n)$ com busca binaria)

Clone (L398-410): Cria copia independente do conjunto

ToString (L431-445): Retorna formato "{q0,q1,q2}"

IsEmpty (L414-418): Verifica se conjunto vazio

Count (L449-453): Retorna numero de elementos

Item (L464-475): Acessa elemento por indice

2.7 Inicializacao da Interface (L488-571)

Referencia: MainForm.pas linhas 488-571

Listing 4: FormCreate

```

1  procedure TFormMain.FormCreate(Sender: TObject);
   begin
       (* Criar listas para alfabeto, estados, etc. *)
       NFAlphabet := TStringList.Create;
       NFAStates := TStringList.Create;
6    NFASinitials := TStringList.Create;
       NFASfinals := TStringList.Create;
       DFAStates := TStringList.Create;
       DFAfinals := TStringList.Create;
       MinDFAStates := TStringList.Create;
11    MinDFAfinals := TStringList.Create;

       (* Carregar arquivos de teste *)
       LoadTestFiles;
       LoadSampleFile;
16  end;
```

2.8 Parser JSON para TXT (L693-918)

O sistema aceita dois formatos: TXT e JSON. Duas funcoes realizam a conversao:

- **ConvertJSONToText (L693-812):** Le arquivo JSON do disco
- **ConvertJSONTextToText (L829-918):** Converte string JSON direta

2.9 LoadSampleFile - Exemplo Padrao (L952-1043)

Referencia: MainForm.pas linhas 952-1043

Carrega arquivo sample_afn.txt se existir, senao cria exemplo hardcoded. O exemplo padrao e um AFN que aceita strings terminadas em "ab".

2.10 btnLoadFileClick - Carregar Arquivo (L1085-1170)

Referencia: MainForm.pas linhas 1085-1170

Event handler do botao "Carregar Arquivo". Abre dialogo nativo (TOpenDialog) para selecionar arquivo .txt ou .json do sistema de arquivos.

Listing 5: btnLoadFileClick

```
procedure TFormMain.btnLoadFileClick(Sender: TObject);
begin
  OpenDialog.Filter := 'Arquivos de automato (*.txt;*.json)|...';
  OpenDialog.DefaultExt := 'txt';

  if OpenDialog.Execute then
  begin
    try
      memoInput.Lines.LoadFromFile(OpenDialog.FileName);
      edtFilePath.Text := OpenDialog.FileName;
    except
      on E: Exception do
        ShowMessage('Erro ao carregar arquivo: ' + E.Message);
    end;
  end;
end;
```

2.11 btnClearClick - Limpar Interface (L1237-1360)

Referencia: MainForm.pas linhas 1237-1360

Limpa completamente a interface e todos os dados, restaurando ao estado inicial.

Operacoes:

- Limpa todos os memos (Input, Output, NFAOutput, MinOutput)
- Limpa todas as TStringList
- Libera arrays de transicoes (SetLength 0)
- Desabilita botoes dependentes
- Redesenha diagramas vazios

2.12 btnRemoveEpsilonClick - Handler Epsilon (L1395-1566)

Referencia: MainForm.pas linhas 1395-1566

Event handler do botao "AFN-epsilon para AFN". Detecta se entrada e JSON e converte temporariamente. Verifica se ha epsilon-transicoes antes de processar. Invoca RemoveEpsilonTransitions se validacao passar.

2.13 btnUseAsInputClick - Copiar Resultado (L1603-1720)

Referencia: MainForm.pas linhas 1603-1720

Copia AFN resultante (de memoNFAOutput) para memoInput, permitindo workflow sequencial.

Listing 6: btnUseAsInputClick

```
procedure TFormMain.btnUseAsInputClick(Sender: TObject);
var i: Integer;
begin
4   memoInput.Lines.Clear;
   memoInput.Lines.Add(NFAAlphabet.CommaText.Replace(',', ' '));
   memoInput.Lines.Add(NFAStates.CommaText.Replace(',', ' '));
   memoInput.Lines.Add(NFAInitials.CommaText.Replace(',', ' '));
   memoInput.Lines.Add(NFAFinals.CommaText.Replace(',', ' '));
9
   for i := 0 to High(NFATransitions) do
       memoInput.Lines.Add(NFATransitions[i].FromState + ' ' +
                           NFATransitions[i].Symbol + ' ' +
                           NFATransitions[i].ToState);
14
   ShowMessage('AFN sem epsilon copiado para a entrada!');
end;
```

2.14 btnConvertClick - Handler Conversao (L1863-2068)

Referencia: MainForm.pas linhas 1863-2068

Event handler principal do botao "AFN para AFD". Detecta JSON e converte automaticamente. Se detectar epsilon-transicoes, executa workflow completo: AFN-epsilon para AFN para AFD. Invoca ConvertAFNtoAFD ao final.

3 Parte 2: Conversao AFN para AFD (L2958-3795)

3.1 Teoria: Construção de Subconjuntos

3.1.1 A Grande Ideia

Princípio Fundamental:

Cada estado do AFD representa um **conjunto de estados** do AFN onde voce poderia estar simultaneamente.

3.1.2 Motivação: Por Que Precisamos Disto?

Problema: AFN pode ter multiplas transicoes com mesmo simbolo.

Exemplo:

- $\delta(q_0, a) = \{q_1, q_2\}$ – AFN vai para q_1 **E** q_2 simultaneamente!
- AFD exige: $\delta(q_0, a) = q_X$ – apenas UM destino

Solução: Criar estado $q_X = \{q_1, q_2\}$ no AFD.

3.1.3 Por Que TStringSet é Essencial?

Problema 1 - Duplicatas:

- Ao calcular $\bigcup_{q \in S} \delta(q, a)$, mesmos estados aparecem multiplas vezes
- TStringSet com Duplicates := dupIgnore elimina automaticamente!

Problema 2 - Comparação de Conjuntos:

- $\{q_0, q_1\}$ e $\{q_1, q_0\}$ são o mesmo estado!
- TStringSet com Sorted := True garante ordem alfabetica
- ToString() sempre retorna mesma string para conjuntos equivalentes

Problema 3 - Performance:

- Busca linear: $O(n)$
- Busca binaria (Sorted=True): $O(\log n)$
- Com milhares de estados, diferenca é gigantesca!

3.2 Estrutura do Algoritmo ConvertAFNtoAFD (L2958-3795)

Referencia: MainForm.pas linhas 2958-3795

Listing 7: Declarações

```

procedure TFormMain.ConvertAFNtoAFD;
var
    (* Variaveis para parsing do AFN *)
    4 Alphabet, States, Initials, Finals: TStringList;
    Transitions: array of TTransition;
    parts: TStringList;
  
```

```

    i, j, k, tStart: Integer;

9    (* Variaveis para construcao do AFD *)
    localDFAStates: TStringList;    (* Estados do AFD *)
    workQ: TStringList;             (* Fila de trabalho BFS *)
    dfaMap: TStringList;            (* Mapa nome para TStringSet *)
    isFinal: TStringList;           (* Estados finais do AFD *)

14    curSet, nextSet, cloneSet: TStringSet;
    sym, key: string;
    localDFATransitions: TTransitionArray;
    hasFinal, hasEpsilon: Boolean;

```

3.3 Declaracao de Variaveis (L2959-2978)

Referencia: MainForm.pas linhas 2959-2978

O algoritmo declara dois grupos de variaveis:

Grupo 1 - Parsing do AFN de entrada:

- Alphabet, States, Initials, Finals: TStringList para armazenar componentes do AFN
- Transitions: Array dinamico de transicoes do AFN
- parts: TStringList para quebrar linhas em tokens
- i, j, k, tStart: Indices para loops

Grupo 2 - Construcao do AFD:

- localDFAStates: Lista de nomes dos estados do AFD (strings como "(q0,q1)")
- workQ: Fila de trabalho BFS (estados que precisam ser processados)
- dfaMap: Mapeamento nome $\rightarrow$ conjunto real (TStringSet)
- isFinal: Lista de estados finais do AFD
- curSet, nextSet, cloneSet: TStringSet para manipular conjuntos
- localDFATransitions: Array de transicoes do AFD

3.4 Limpeza e Inicializacao (L2980-3005)

Referencia: MainForm.pas linhas 2980-3005

Por que limpar? Evitar dados misturados de conversoes anteriores.

Procedimento:

1. Verifica se objetos existem com Assigned()
2. Libera memoria com Free()
3. Cria novas instancias vazias

Estruturas recriadas:

- NFAlphabet, NFASStates, NFAlnitials, NFAlnfinals (para guardar AFN)
- DFASStates, DFAlnfinals (para construir AFD)

3.5 Validacao de Entrada (L3007-3018)

Referencia: MainForm.pas linhas 3007-3018

Verifica se entrada tem pelo menos 4 linhas:

1. Alfabeto
2. Estados
3. Iniciais
4. Finais

Se invalida: Mostra mensagem e aborta com `Exit`.

3.6 Parse do Alfabeto (L3033-3058)

Referencia: MainForm.pas linhas 3033-3058

Objetivo: Ler simbolos do alfabeto da linha 0.

Logica:

```
parts.DelimitedText := Trim(memoInput.Lines[0]);  
2 for i := 0 to parts.Count - 1 do  
    if (simbolo nao e epsilon) then  
        Alphabet.Add(parts[i]);
```

Filtragem de epsilon: AFD nao pode ter ϵ -transicoes, entao filtra:

- 'ε' (Unicode)
- 'epsilon' (palavra)
- 'e' (abreviacao)
- '&' (alternativa)

3.7 Parse dos Estados (L3067-3075)

Referencia: MainForm.pas linhas 3067-3075

Objetivo: Ler lista de estados da linha 1.

Cada estado e uma string (ex: "q0", "q1", "q2").

Armazena em:

- **States:** Lista local
- **NFAStates:** Campo da classe (usado depois)

3.8 Parse dos Estados Iniciais (L3084-3092)

Referencia: MainForm.pas linhas 3084-3092

Objetivo: Ler estados iniciais da linha 2.

Importante: AFN pode ter multiplos iniciais!

No AFD: estado inicial = **conjunto** dos iniciais do AFN.

3.9 Parse dos Estados Finais (L3101-3109)

Referencia: MainForm.pas linhas 3101-3109

Objetivo: Ler estados finais da linha 3.

Regra para AFD: Estado do AFD e final $\Leftrightarrow$ contem algum estado final do AFN.

3.10 Parse das Transicoes (L3130-3177)

Referencia: MainForm.pas linhas 3130-3177

Formato esperado: origem simbolo destino

Algoritmo:

```
1 for i := 4 to memoInput.Lines.Count - 1 do
  parts.DelimitedText := linha;
  if parts.Count >= 3 then
    Transicao := (parts[0], parts[1], parts[2]);
```

Deteccao de epsilon: Se encontrar ε -transicao, marca flag hasEpsilon.

3.11 Aviso de Epsilon (L3179-3191)

Referencia: MainForm.pas linhas 3179-3191

Se hasEpsilon = True:

- Mostra dialogo de aviso
- Orienta usuario: $\text{AFN-}\varepsilon \rightarrow \text{AFN} \rightarrow \text{AFD}$
- Continua execucao (resultado pode estar incorreto)

Por que avisar? AFD nao pode ter ε por definicao!

3.11.1 Filosofia de Validacao: Avisar vs Bloquear

Alternativa 1 - Bloquear execucao:

```
1 if hasEpsilon then
  begin
    ShowMessage('ERRO: AFD nao aceita epsilon!');
    Exit;  (* Aborta conversao *)
  end;
```

Alternativa 2 - Avisar e continuar (implementado):

```
if hasEpsilon then
  ShowMessage('AVISO: Entrada tem epsilon. Use RemoveEpsilon primeiro
  .');
(* Continua execucao *)
```

Por que escolhemos Alternativa 2?

1. **Flexibilidade:** Usuario pode querer testar mesmo assim
2. **Debugging:** Permite ver resultado incorreto para comparacao
3. **Workflow educacional:** Mostra o que acontece com epsilon

4. **Nao-destrutivo:** Nao perde trabalho do usuario

Trade-off:

- **Pro:** Usuario tem controle e visibilidade
- **Contra:** Usuario pode ignorar aviso e obter resultado errado

Decisao de design: Em ferramenta educacional, preferimos avisar e ensinar ao inves de bloquear.

3.12 Logging da Entrada (L3206-3217)

Referencia: MainForm.pas linhas 3206-3217

Imprime no console (WriteLn) o AFN de entrada para debugging:

- Alfabeto
- Estados
- Iniciais
- Finais
- Lista de transicoes

3.13 Criacao das Estruturas BFS (L3251-3257)

Referencia: MainForm.pas linhas 3251-3257

Estruturas criadas:

localDFASStates	Lista de estados do AFD (nomes)
workQ	Fila BFS (estados a processar)
dfaMap	Mapa nome $\rightarrow$ TStringSet
isFinal	Estados finais do AFD

Importante: dfaMap.OwnsObjects := True
 $\rightarrow$ libera TStringSets automaticamente!

3.13.1 Por Que BFS e Nao DFS?

Razao 1 - Ordem Natural:

- BFS processa estados por "distancia" do inicial
- Resultado mais intuitivo para visualizacao
- Estados principais aparecem primeiro no output

Razao 2 - Memoria Controlada:

- DFS: Pilha pode crescer exponencialmente em automatos complexos
- BFS: Fila cresce de forma mais previsivel
- Importante para automatos com milhares de estados

Razao 3 - Deteccao de Loops:

- BFS detecta estados repetidos naturalmente
- DFS pode ficar preso em ciclos se nao implementado corretamente

3.13.2 Por Que OwnsObjects = True?

Problema sem OwnsObjects:

```
dfaMap.AddObject('{q0,q1}', set1);
2 dfaMap.AddObject('{q2,q3}', set2);
dfaMap.Free; (* Libera dfaMap mas NAO os TStringSets! *)
(* MEMORY LEAK! set1 e set2 ainda estao na memoria! *)
```

Solucao com OwnsObjects:

```
1 dfaMap.OwnsObjects := True;
dfaMap.AddObject('{q0,q1}', set1);
dfaMap.Free; (* Libera dfaMap E todos os TStringSets! *)
(* Sem memory leak! *)
```

Por que isso importa?

- AFD com 100 estados = 100 TStringSet para liberar manualmente!
- Esqueceu um Free()? Memory leak.
- OwnsObjects = True → gerenciamento automatico!

3.14 Estado Inicial do AFD (L3273-3283)

Referencia: MainForm.pas linhas 3273-3283

Regra: Estado inicial do AFD = **conjunto** dos iniciais do AFN.

Algoritmo:

```
1 curSet := TStringSet.Create;
for i := 0 to Initials.Count - 1 do
    curSet.Add(Initials[i]);

localDFAStates.Add(GetSetName(curSet));
6 workQ.Add(GetSetName(curSet));
dfaMap.AddObject(GetSetName(curSet), curSet);
```

Exemplo: Se AFN tem iniciais (q0, q1) → estado inicial do AFD = "{q0,q1}"

3.15 Loop Principal BFS (L3296-3540)

Referencia: MainForm.pas linhas 3296-3540

Algoritmo de Busca em Largura:

```
while workQ.Count > 0 do
    (* 1. Remover estado da fila *)
3    curSet := ProximoEstado();

    (* 2. Verificar se e final *)
    if curSet contem algum final do AFN then
        Marcar como final;

8
    (* 3. Para cada simbolo do alfabeto *)
    for cada simbolo do
        nextSet := CalcularDestinos(curSet, simbolo);
        CriarTransicao(curSet, simbolo, nextSet);

13
```



```

(* 4. Se nextSet e novo, adicionar a fila *)
if nextSet e novo then
    workQ.Add(nextSet);
end;

```

Invariante: Todos estados em `localDFASStates` foram ou serao processados.

Terminacao: Numero de estados $\leq 2^n$ (finito).

3.15.1 Exemplo Pratico Passo-a-Passo

AFN de entrada:

- Estados: $\{q_0, q_1, q_2\}$
- Alfabeto: $\{a, b\}$
- Iniciais: $\{q_0\}$
- Finais: $\{q_2\}$
- Transicoes: $(q_0, a, q_0), (q_0, a, q_1), (q_1, b, q_2)$

Execucao do BFS:

Iteracao 1:

1. `workQ = [{"q0"}]`, remove "`{q0}`"
2. Nao e final (`q0` nao e final)
3. Com 'a': destinos = $\{q_0, q_1\} \rightarrow$ transicao ($\{q_0\}$, a, $\{q_0, q_1\}$)
4. Com 'b': destinos = $\{\} \rightarrow$ transicao ($\{q_0\}$, b, $\{\}$)
5. Novo estado descoberto: "`{q0,q1}`" $\rightarrow$ adiciona a fila
6. `workQ = [{"q0,q1"}]`

Iteracao 2:

1. `workQ = [{"q0,q1"}]`, remove "`{q0,q1}`"
2. Nao e final (nem `q0` nem `q1` sao finais)
3. Com 'a': destinos = $\{q_0, q_1\} \rightarrow$ transicao ($\{q_0, q_1\}$, a, $\{q_0, q_1\}$)
4. Com 'b': destinos = $\{q_2\} \rightarrow$ transicao ($\{q_0, q_1\}$, b, $\{q_2\}$)
5. Novo estado descoberto: "`{q2}`" $\rightarrow$ adiciona a fila
6. `workQ = [{"q2"}]`

Iteracao 3:

1. `workQ = [{"q2"}]`, remove "`{q2}`"
2. E final! (`q2` e final)

3. Com 'a': destinos = {} → transicao ({q2}, a, {})

4. Com 'b': destinos = {} → transicao ({q2}, b, {})

5. Nenhum novo estado

6. workQ = []

Fim! AFD resultante tem 3 estados: {q0}, {q0,q1}, {q2}

3.16 Remocao da Fila (L3308-3311)

Referencia: MainForm.pas linhas 3308-3311

```
key := workQ[0];
workQ.Delete(0);
3 curSet := TStringSet(dfaMap.Objects[...]);
```

FIFO: Remove do inicio (BFS), nao do fim.

3.17 Verificacao de Estado Final (L3326-3330)

Referencia: MainForm.pas linhas 3326-3330

Regra: Estado AFD e final ⇔ curSet contem algum final do AFN.

```
hasFinal := False;
2 for i := 0 to Finals.Count - 1 do
    if curSet.Contains(Finals[i]) then
        hasFinal := True;
if hasFinal then isFinal.Add(key);
```

Justificativa: Se caminho leva a estado final do AFN, o conjunto todo e final.

3.18 Loop de Simbolos (L3351-3525)

Referencia: MainForm.pas linhas 3351-3525

Para cada simbolo do alfabeto, calcula proximos estados:

Algoritmo:

```
for i := 0 to Alphabet.Count - 1 do
    sym := Alphabet[i];
    nextSet := TStringSet.Create;

5    (* Uniao dos destinos *)
    for cada q em curSet do
        for cada transicao (q, sym, r) do
            nextSet.Add(r);

10    (* Criar transicao AFD *)
    Transicao := (curSet, sym, nextSet);
```

Formula: $\delta_{AFD}(S, a) = \bigcup_{q \in S} \delta_{AFN}(q, a)$

3.19 Calculo de Destinos (L3356-3373)

Referencia: MainForm.pas linhas 3356-3373

```

4  for j := 0 to curSet.Count - 1 do
    for k := 0 to High(Transitions) do
      if (Transitions[k].FromState = curSet.Item(j))
        and (Transitions[k].Symbol = sym) then
        nextSet.Add(Transitions[k].ToState);

```

Complexidade: $O(|curSet| \cdot m)$ onde m = transicoes do AFN.

3.20 Tratamento de Conjunto Vazio (L3375-3379)

Referencia: MainForm.pas linhas 3375-3379

Se nextSet.IsEmpty:

- AFD vai para estado de erro "{}"
- Estado de erro NAO e adicionado ao AFD (dead state implicito)

Exemplo: (q0) com 'b' e nenhuma transicao existe $\rightarrow$ "{}"

3.20.1 Por Que Nao Adicionar Estado de Erro?

Razao 1 - Tamanho do AFD:

- Estado de erro tem transicoes para si mesmo com TODOS os simbolos
- Em alfabeto grande, adiciona muitas transicoes inuteis
- Exemplo: alfabeto de 26 letras = 26 transicoes extras!

Razao 2 - Visualizacao:

- Estado de erro polui diagrama
- Usuario sabe: "sem transicao = rejeita"
- Mais limpo omitir

Razao 3 - Minimizacao:

- Algoritmo de minimizacao remove estado de erro de qualquer forma
- Por que adicionar so para remover depois?

Trade-off:

- **Com estado erro:** AFD teoricamente completo
- **Sem estado erro:** AFD parcial (mas funcionalmente identico)

Decisao de implementacao: Omitir estado de erro para AFD mais limpo e legivel.

3.21 Registro de Transicao (L3395-3409)

Referencia: MainForm.pas linhas 3395-3409

Expande array dinamico e adiciona transicao:

```
SetLength(localDFATransitions, Length(...) + 1);
localDFATransitions[High(...)].FromState := curSet;
localDFATransitions[High(...)].Symbol := sym;
localDFATransitions[High(...)].ToState := nextSet;
```

Armazena em DOIS lugares:

- localDFATransitions (variavel local)
- DFATransitions (campo da classe)

3.22 Descoberta de Novos Estados (L3426-3439)

Referencia: MainForm.pas linhas 3426-3439

Condicao: Estado e novo SE:

- dfaMap.IndexOf(key) = -1 (nao esta no mapa)
- key $\neq$ '{ }' (nao e estado de erro)

Acoes:

```
1 cloneSet := nextSet.Clone;
dfaMap.AddObject(key, cloneSet);
localDFASStates.Add(key);
workQ.Add(key); (* Adicionar a fila! *)
```

Por que Clone? nextSet sera liberado no finally. Precisamos de copia!

3.22.1 Gerenciamento de Memoria Detalhado

Problema sem Clone():

```
1 nextSet := TStringSet.Create; (* Cria nextSet *)
dfaMap.AddObject(key, nextSet); (* Adiciona ao mapa *)
nextSet.Free; (* ERRO! dfaMap aponta para memoria liberada! *)
```

Por que nextSet precisa ser liberado?

- Criado dentro do loop: nextSet := TStringSet.Create
- Sem Free(): memory leak em CADA iteracao!
- AFD com 100 estados = 100 conjuntos nao liberados

Solucao: Clone antes de adicionar

```
nextSet := TStringSet.Create; (* Conjunto temporario *)
2 cloneSet := nextSet.Clone; (* Copia independente *)
dfaMap.AddObject(key, cloneSet); (* Adiciona copia ao mapa *)
nextSet.Free; (* Libera temporario *)
(* cloneSet sera liberado quando dfaMap for destruido *)
```

Ciclo de vida dos objetos:

1. `nextSet`: Vive apenas durante uma iteracao do loop
2. `cloneSet`: Vive ate `dfaMap.Free`
3. `dfaMap`: Liberado no bloco `finally` ao fim da funcao

3.22.2 Por Que `IndexOf` ao Inves de `Contains`?

`TStringList.IndexOf` vs busca manual:

- `IndexOf(key)` retorna indice (-1 se nao existe)
- Com `Sorted := True`, usa busca binaria $O(\log n)$
- Loop manual seria $O(n)$
- Com 1000 estados: $\log_2(1000) \approx 10$ vs 1000 comparacoes!

Por que comparar com -1?

- `IndexOf` retorna -1 quando nao encontra
- Indices validos: 0, 1, 2, ..., `Count-1`
- `IndexOf(key) = -1  $\equiv$  "key nao existe"`

3.23 Formatacao da Saida (L3549-3638)

Referencia: `MainForm.pas` linhas 3549-3638

Apresenta resultado no `memoOutput` com:

- Cabecalho decorativo
- Alfabeto
- Lista de estados do AFD
- Estado inicial (sempre o primeiro)
- Estados finais
- Lista de transicoes

Tambem copia para campos da classe: `DFAStates`, `DFAInitial`, `DFAFinals`.

3.23.1 Por Que Dois Destinos para o Resultado?

Destino 1 - `memoOutput` (UI):

- Usuario ve resultado imediatamente
- Formato texto legivel
- Pode copiar/colar
- Pode salvar em arquivo

Destino 2 - Campos da classe (DFASStates, etc):

- Proxima etapa (minimizacao) precisa dos dados
- Funcao DrawAutomaton precisa desenhar diagrama
- Botao "Usar como entrada" precisa copiar

Exemplo de pipeline:

1. Conversao AFN $\rightarrow$ AFD: resultado em DFASStates
2. Usuario clica "Minimizar": le DFASStates, escreve MinDFASStates
3. Usuario clica "Desenhar": le DFASStates, desenha diagrama
4. Usuario clica "Exportar": le memoOutput, salva arquivo

Por que nao armazenar APENAS em UI?

- Parse de texto e custoso ($O(n \cdot m)$)
- Parsing pode introduzir erros
- Estruturas de dados ja existem, reaproveitar!

3.23.2 Formato de Estados do AFD

Notacao escolhida: "{q0,q1,q2}"

Por que chaves?

- Notacao matematica padrao para conjuntos
- Usuario reconhece: "isso e um conjunto!"
- Diferencia de estados simples: "q0" vs "{q0}"

Por que virgulas sem espacos?

- ToString() de TStringSet usa CommaText
- CommaText automaticamente coloca virgulas
- Formato compacto: "{q0,q1}" vs "{q0, q1}"

Alternativas consideradas:

- "(q0,q1)-- parece tupla, nao conjunto
- "[q0,q1]-- parece array
- "q0q1-- nao distingue "q01" de "q0,q1"

3.24 Habilidade da Interface (L3640-3647)

Referencia: MainForm.pas linhas 3640-3647

```
btnMinimize.Enabled := True;
PageControl2.ActivePage := TabOutput;
PaintBoxNFA.Invalidate;
PaintBoxDFA.Invalidate;
```

Acoes:

- Habilita botao "Minimizar AFD"
- Muda para aba de resultado
- Redesenha diagramas

3.25 Limpeza de Memoria (L3650-3665)

Referencia: MainForm.pas linhas 3650-3665

Libera TODAS estruturas locais no finally:

```
1 localDFAStates.Free;
  workQ.Free;
  dfaMap.Free; (* Libera TStringSets tambem! *)
  isFinal.Free;
  Alphabet.Free;
6 States.Free;
  Initials.Free;
  Finals.Free;
  parts.Free;
```

IMPORTANTE: dfaMap.OwnsObjects = True → Free() tambem libera todos os TStringSet!

3.26 Paint Handlers - Desenho de Diagramas (L3698-3792)

Referencia: MainForm.pas linhas 3698-3792

O codigo implementa 4 event handlers para desenhar diagramas de automatos.

3.26.1 PaintBoxEpsilonNFAPaint (L3698-3710)

Proposito: Desenhar AFN- ϵ no painel grafico.

Quando e chamado:

- Primeira exibicao da aba
- Apos Invalidate()
- Redimensionamento da janela

```
1 procedure TFormMain.PaintBoxEpsilonNFAPaint(Sender: TObject);
  begin
    if not Assigned(EpsilonNFASStates) then Exit;
    DrawAutomaton(PaintBoxEpsilonNFA.Canvas,
                  EpsilonNFASStates, EpsilonNFAInitials,
6                  EpsilonNFAFinals, EpsilonNFATransitions);
  end;
```

Validacao: Se nao ha dados, sai sem desenhar.

3.26.2 PaintBoxNFAPaint (L3712-3718)

Proposito: Desenhar AFN (sem epsilon) no painel.

Mesmo padrao do anterior, mas usa estruturas NFAXXX.

3.26.3 PaintBoxDFAPaint (L3720-3741)

Proposito: Desenhar AFD no painel.

Diferenca importante: AFD tem apenas 1 estado inicial!

```
procedure TFormMain.PaintBoxDFAPaint(Sender: TObject);
var Initials: TStringList;
3 begin
    if not Assigned(DFAStates) then Exit;

    Initials := TStringList.Create;
    try
8      if DFAStates.Count > 0 then
        Initials.Add(DFAStates[0]); (* Primeiro = inicial *)

        DrawAutomaton(PaintBoxDFA.Canvas, DFAStates,
                      Initials, DFAFinals, DFATransitions);
13    finally
        Initials.Free; (* Liberar memoria! *)
    end;
end;
```

Por que TStringList temporaria? DrawAutomaton espera TStringList de iniciais. AFD tem string unica, precisa converter.

3.26.4 PaintBoxMinDFAPaint (L3743-3764)

Proposito: Desenhar AFD Minimizado no painel.

Mesmo padrao do PaintBoxDFAPaint, mas usa:

- MinDFAStates
- MinDFAInitial
- MinDFAFinals
- MinDFATransitions

3.27 Complexidade do Algoritmo Completo

Analise de cada parte:

Onde:

- n = numero de estados do AFN
- m = numero de transicoes do AFN
- $|\Sigma|$ = tamanho do alfabeto

- $|S|$ = tamanho medio dos conjuntos
- $|F|$ = numero de estados finais

Explosao exponencial: No pior caso, AFD pode ter 2^n estados!

Exemplo: AFN com 10 estados $\rightarrow$ AFD com ate 1024 estados.

3.28 Mapa Completo de Funcoes

4 Conclusao

4.1 Resumo das Responsabilidades

Henrique implementou:

1. Estruturas de Dados Fundamentais

- TTransition (record de transicao)
- TStringSet (classe de conjuntos)
- Variaveis globais para AFN/AFD

2. Interface e Carregamento

- Inicializacao do formulario
- Scan de arquivos de teste
- Parser JSON para TXT

3. Algoritmo Principal: AFN para AFD

- Construcao de subconjuntos
- Determinacao de estados finais
- Geracao de transicoes deterministicas
- Logging e interface automatica

4.2 Pontos-Chave para Apresentacao

1. **TStringSet** e essencial - evita duplicatas nos conjuntos
2. **Cada estado do AFD = conjunto de estados do AFN**
3. **Loop principal:** Para cada conjunto nao processado, para cada simbolo, calcule TODOS os destinos possiveis
4. **Estado final:** Se o conjunto contem pelo menos um estado final do AFN
5. **Complexidade:** Exponencial no pior caso (2^n), mas tipico e linear
6. **Parse robusto:** Aceita JSON e TXT, converte automaticamente
7. **Validacao completa:** Detecta epsilon e orienta usuario sobre workflow correto

4.3 Complexidade do Algoritmo

Funcao	Linhas	Descricao
PARTE 1: Estruturas e Interface (L1-2254)		
TTransition (type)	39-43	Record de transicao
TTransitionArray (type)	49	Array dinamico
Variaveis globais	135-165	AFN-e, AFN, AFD, MinDFA
TStringSet (class)	299-311	Declaracao da classe
TStringSet.Create	333-337	Construtor
TStringSet.Destroy	339-343	Destrutor
TStringSet.Add	360-367	Adicionar elemento
TStringSet.Contains	377-381	Verificar existencia
TStringSet.Clone	398-410	Clonar conjunto
TStringSet.IsEmpty	414-418	Verificar vazio
TStringSet.ToString	431-445	Converter para string
TStringSet.Count	449-453	Numero de elementos
TStringSet.Item	464-475	Acessar por indice
FormCreate	488-571	Inicializar interface
LoadTestFiles	586-643	Scan arquivos testes/
OnTestFileSelected	645-679	Handler ComboBox
ConvertJSONToText	693-812	Converter arquivo JSON
ConvertJSONTextToText	829-918	Converter string JSON
LoadSampleFile	952-1043	Carregar exemplo
btnLoadFileClick	1085-1170	Handler carregar arquivo
btnClearClick	1237-1360	Limpar interface
btnRemoveEpsilonClick	1395-1566	Handler remocao epsilon
btnUseAsInputClick	1603-1720	Copiar AFN para input
btnConvertClick	1863-2068	Handler conversao AFN para AFD
PARTE 2: Algoritmo AFN para AFD (L2958-3795)		
ConvertAFNtoAFD (inicio)	2958-3020	Declaracoes e variaveis
Limpeza e criacao	2970-3010	Liberar e criar estruturas
Validacao entrada	3011-3018	Verificar 4+ linhas
Parse alfabeto	3040-3058	Ler linha 0 (simbolos)
Parse estados	3060-3068	Ler linha 1 (estados)
Parse iniciais	3070-3078	Ler linha 2 (iniciais)
Parse finais	3080-3088	Ler linha 3 (finais)
Parse transicoes	3090-3220	Ler linhas 4+ (transicoes)
Logging entrada	3222-3235	Exibir AFN no console
Inicializacao BFS	3237-3258	Criar estruturas AFD
Estado inicial AFD	3260-3270	Conjunto dos iniciais
Loop principal BFS	3272-3580	Subset construction
Determinar finais	3360-3380	Marcar estados finais
Gerar transicoes	3390-3500	Para cada simbolo
Descobrir novos estados	3510-3550	Adicionar a fila
Formatacao resultado	3582-3680	Texto para memoOutput
Atualizar interface	3682-3696	Habilitar botoes
PaintBoxEpsilonNFAPaint	3698-3723	Desenhar AFN-epsilon
PaintBoxNFAPaint	3725-3737	Desenhar AFN
PaintBoxDFAPaint	3739-3769	Desenhar AFD
PaintBoxMinDFAPaint	3771-3792	Desenhar MinDFA

Tabela 1: Indice Completo com Linhas Exatas

Operacao	Complexidade
Parse entrada	$O(n + m)$
Estado inicial	$O(n_{init})$
Loop principal BFS	$O(2^n)$ iteracoes
Por cada estado	$O(\Sigma \cdot S \cdot m)$
Verificacao final	$O(S \cdot F)$
Calculo destinos	$O(S \cdot m)$
Total pior caso	$O(2^n \cdot \Sigma \cdot n \cdot m)$

Tabela 2: Complexidade do Subset Construction

Funcao	Linhas	Descricao
PARTE 1: Estruturas e Interface		
TTransition	39-43	Record de transicao
TStringSet.Create	333-337	Construtor
TStringSet.Destroy	339-343	Destrutor
FormCreate	488-571	Inicializar interface
LoadTestFiles	586-643	Scan arquivos testes/
ConvertJSONToText	693-812	Converter arquivo JSON
ConvertJSONTextToText	829-918	Converter string JSON
LoadSampleFile	952-1043	Carregar exemplo
btnLoadFileClick	1085-1170	Handler carregar arquivo
btnClearClick	1237-1360	Limpar interface
btnRemoveEpsilonClick	1395-1566	Handler remocao epsilon
btnUseAsInputClick	1603-1720	Copiar AFN para input
btnConvertClick	1863-2068	Handler conversao AFN para AFD
GetSetName	2006-2058	TStringSet para String
ComputeEpsilonClosure	2109-2300	Calcular epsilon-fecho
PARTE 2: Algoritmo AFN para AFD		
ConvertAFNtoAFD (inicio)	2958-3020	Declaracoes e init
Parse alfabeto	3040-3055	Ler linha 0
Parse estados	3057-3065	Ler linha 1
Parse iniciais	3067-3075	Ler linha 2
Parse finais	3077-3085	Ler linha 3
Parse transicoes	3150-3220	Ler linhas 4+
Inicializacao	3221-3270	Criar estruturas AFD
Loop principal	3271-3600	BFS subset construction
Formatacao saida	3601-3720	Gerar memoOutput
PaintBoxEpsilonNFAPaint	3721-3735	Desenhar AFN-epsilon
PaintBoxNFAPaint	3737-3745	Desenhar AFN
PaintBoxDFAPaint	3747-3765	Desenhar AFD
PaintBoxMinDFAPaint	3767-3785	Desenhar MinDFA

Tabela 3: Indice Completo de Funcoes - Secoes do Henrique

Operacao	Complexidade	Motivo
Estados do AFD	$O(2^n)$	No pior caso, todos subconjuntos
Loop principal	$O(estados \cdot \Sigma)$	Para cada estado/simbolo
Busca de transicoes	$O(m)$	$m =$ transicoes do AFN
Total	$O(2^n \cdot \Sigma \cdot m)$	Pior caso exponencial

Tabela 4: Analise de Complexidade